

# HUSSEIN MBOYA TOURS

Professional Code, Security, Architecture, Software Quality & Tour-Management System  
Audit

Assessment date: 16 August 2026

Assessment type: Source-code and configuration review of supplied MERN/Vite  
application

Scope: Backend API, authentication, RBAC, booking/tour operations, payments, uploads,  
sockets, AI, data protection, frontend security, reliability, DevSecOps and operational  
readiness

IMPORTANT: This is a source-code audit, not a certification, penetration test, production configuration assessment, payment-provider attestation, or legal opinion. Findings marked open require implementation and verification.

# Document control & executive position

## Assessment conclusion

The supplied application is a substantial and feature-rich travel/tour management platform with a broad MERN architecture: customer booking, tours, destinations, agents, guides, drivers, vehicles, finance, payments, coupons, invoices, analytics, notifications, AI features and administrative operations. The codebase contains several strong foundations—Helmet, CORS allowlisting, JWT signing, bcrypt password hashing, Mongoose validation, rate limiting, Cloudinary upload limits, soft deletion and multiple RBAC layers.

## Security position

The most important risk is not a single injection bug; it is authorization consistency across a large route/controller surface. The codebase has multiple authorization implementations and legacy role aliases. Several operational endpoints were directly mounted without authentication in the supplied build. Those routes have been hardened in the audit copy. Socket identity handling was also hardened.

## Business position

Tour management has safety-critical business invariants: capacity, booking state, payment state, refund state, assignment, traveler identity, emergency information, vehicle capacity and staff availability. These must be treated as transactional business rules, not only CRUD fields. The audit therefore evaluates both conventional web security and operational correctness.

## Release recommendation

Do not treat the current state as production-ready solely because the supplied remediation build passes syntax checks. Before go-live, close the open High findings, run authenticated API penetration tests, test payment callbacks with the real providers, verify backups/restores, and establish privacy/retention procedures.

# Methodology — Scope and evidence

## Assessment notes

The assessment used the supplied ZIP as the primary evidence source. Static inspection covered the server application, controllers, routes, models, middleware, configuration, services, socket layer and React client. Package manifests and lockfiles were reviewed for dependency versions. Runtime execution was not possible against the user's production infrastructure, and the package registry audit endpoint was unavailable during the review window.

# Methodology — Methodology and standards

## Assessment notes

The assessment is mapped conceptually to OWASP ASVS 5.0, OWASP API Security Top 10 2023, secure SDLC principles, common cloud/application security controls and tour-management business risks. OWASP describes ASVS as a basis for testing web application technical security controls. API Top 10 emphasizes broken authorization, resource consumption, sensitive business flows, SSRF and security misconfiguration.

# Methodology — Severity model

## Assessment notes

Critical: likely compromise, major unauthorized control or severe confidentiality/integrity impact. High: material security or business risk that should block a normal production release. Medium: meaningful weakness requiring planned remediation. Low: hardening, maintainability or defense-in-depth. Informational: architectural or process recommendation. Severity reflects this application's travel/payment/PII context.

# Methodology — Limitations

## Assessment notes

No production credentials, live payment gateway, Cloudinary account, SMS provider, MongoDB instance, browser session, WAF, CDN, CI/CD pipeline or hosting configuration were supplied. Consequently, claims about live TLS, database network controls, webhook signatures, cloud IAM, backup recoverability and real-world exploitability remain verification items.

# Methodology — Audit artifacts

## Assessment notes

The returned ZIP contains the original application plus an AUDIT directory. The audit report is this PDF; remediation notes describe the changes made in the supplied copy. The application code was syntax-checked after the security patches on the server JavaScript surface. Client JSX requires the project build/lint toolchain for full validation.

# F-001 — Public database endpoint exposed

## Severity and status

**Critical.** Remediated in supplied build.

## Evidence

server/routes/databaseRoutes.js + server/server.js

## Assessment

Unauthenticated callers could query database status and future implementation risk. Hardened with protect + system.database authorization.

## Tour-management impact

For a tour platform, authorization failures can expose passenger records, modify itineraries, alter pricing, change booking state, assign guides/drivers/vehicles, manipulate payment state or reveal operational information. Controls therefore need both identity authorization and business-object authorization.

## Acceptance criteria

The control should be enforced server-side, default-deny, covered by an automated test, logged without sensitive payloads, and verified using both an allowed-role test and a denied-role/object test.



# **F-001 — remediation & verification**

## **Recommended design**

Implement and verify the control with a narrow allowlist and explicit business rules. Avoid relying on UI visibility. Add automated regression tests for anonymous, customer, agent, manager, guide, driver, admin and super-admin personas as appropriate. The supplied audit build already contains a targeted remediation for this finding; the change should still be exercised in integration tests before deployment.

## **Verification procedure**

1) Reproduce the original access path from an unauthenticated client where relevant. 2) Repeat with a least-privileged authenticated user. 3) Repeat with the intended privileged role. 4) Test an invalid object ID and another user's object ID. 5) Confirm the response does not disclose internal state. 6) Confirm an audit/security event exists where the action is security-sensitive.

## **Regression risk**

Security patches can break legacy frontend flows. Keep compatibility aliases only when their authorization policy is identical. Do not restore an insecure route merely to satisfy an old client; update the client instead.

## **Operational control**

Add the check to CI and to the release gate. A High finding should not be reopened without a documented compensating control and owner.

## **F-002 — System settings route exposed without authentication**

### **Severity and status**

**Critical.** Remediated in supplied build.

### **Evidence**

server/routes/settingsRoutes.js + server/server.js

### **Assessment**

The same router was mounted directly under /api/settings and had no middleware. Hardened with protect + settings.manage.

### **Tour-management impact**

For a tour platform, authorization failures can expose passenger records, modify itineraries, alter pricing, change booking state, assign guides/drivers/vehicles, manipulate payment state or reveal operational information. Controls therefore need both identity authorization and business-object authorization.

### **Acceptance criteria**

The control should be enforced server-side, default-deny, covered by an automated test, logged without sensitive payloads, and verified using both an allowed-role test and a denied-role/object test.

## **F-002 — remediation & verification**

### **Recommended design**

Implement and verify the control with a narrow allowlist and explicit business rules. Avoid relying on UI visibility. Add automated regression tests for anonymous, customer, agent, manager, guide, driver, admin and super-admin personas as appropriate. The supplied audit build already contains a targeted remediation for this finding; the change should still be exercised in integration tests before deployment.

### **Verification procedure**

1) Reproduce the original access path from an unauthenticated client where relevant. 2) Repeat with a least-privileged authenticated user. 3) Repeat with the intended privileged role. 4) Test an invalid object ID and another user's object ID. 5) Confirm the response does not disclose internal state. 6) Confirm an audit/security event exists where the action is security-sensitive.

### **Regression risk**

Security patches can break legacy frontend flows. Keep compatibility aliases only when their authorization policy is identical. Do not restore an insecure route merely to satisfy an old client; update the client instead.

### **Operational control**

Add the check to CI and to the release gate. A High finding should not be reopened without a documented compensating control and owner.

## **F-003 — System health leaked environment/runtime internals**

### **Severity and status**

**High.** Remediated in supplied build.

### **Evidence**

server/routes/systemHealthRoutes.js; server/routes/index.js

### **Assessment**

Responses exposed Node version, environment, memory, OS and architecture. Hardened to minimal DB/application health and protected admin access.

### **Tour-management impact**

For a tour platform, authorization failures can expose passenger records, modify itineraries, alter pricing, change booking state, assign guides/drivers/vehicles, manipulate payment state or reveal operational information. Controls therefore need both identity authorization and business-object authorization.

### **Acceptance criteria**

The control should be enforced server-side, default-deny, covered by an automated test, logged without sensitive payloads, and verified using both an allowed-role test and a denied-role/object test.

## **F-003 — remediation & verification**

### **Recommended design**

Implement and verify the control with a narrow allowlist and explicit business rules. Avoid relying on UI visibility. Add automated regression tests for anonymous, customer, agent, manager, guide, driver, admin and super-admin personas as appropriate. The supplied audit build already contains a targeted remediation for this finding; the change should still be exercised in integration tests before deployment.

### **Verification procedure**

1) Reproduce the original access path from an unauthenticated client where relevant. 2) Repeat with a least-privileged authenticated user. 3) Repeat with the intended privileged role. 4) Test an invalid object ID and another user's object ID. 5) Confirm the response does not disclose internal state. 6) Confirm an audit/security event exists where the action is security-sensitive.

### **Regression risk**

Security patches can break legacy frontend flows. Keep compatibility aliases only when their authorization policy is identical. Do not restore an insecure route merely to satisfy an old client; update the client instead.

### **Operational control**

Add the check to CI and to the release gate. A High finding should not be reopened without a documented compensating control and owner.

# F-004 — Super-admin tools routes lacked authorization

## Severity and status

**Critical.** Remediated in supplied build.

## Evidence

server/routes/superAdminToolsRoutes.js

## Assessment

Database/settings tool endpoints were directly callable. Added protect + system.security.

## Tour-management impact

For a tour platform, authorization failures can expose passenger records, modify itineraries, alter pricing, change booking state, assign guides/drivers/vehicles, manipulate payment state or reveal operational information. Controls therefore need both identity authorization and business-object authorization.

## Acceptance criteria

The control should be enforced server-side, default-deny, covered by an automated test, logged without sensitive payloads, and verified using both an allowed-role test and a denied-role/object test.

## **F-004 — remediation & verification**

### **Recommended design**

Implement and verify the control with a narrow allowlist and explicit business rules. Avoid relying on UI visibility. Add automated regression tests for anonymous, customer, agent, manager, guide, driver, admin and super-admin personas as appropriate. The supplied audit build already contains a targeted remediation for this finding; the change should still be exercised in integration tests before deployment.

### **Verification procedure**

1) Reproduce the original access path from an unauthenticated client where relevant. 2) Repeat with a least-privileged authenticated user. 3) Repeat with the intended privileged role. 4) Test an invalid object ID and another user's object ID. 5) Confirm the response does not disclose internal state. 6) Confirm an audit/security event exists where the action is security-sensitive.

### **Regression risk**

Security patches can break legacy frontend flows. Keep compatibility aliases only when their authorization policy is identical. Do not restore an insecure route merely to satisfy an old client; update the client instead.

### **Operational control**

Add the check to CI and to the release gate. A High finding should not be reopened without a documented compensating control and owner.

## **F-005 — Socket authentication relied on client-supplied user ID**

### **Severity and status**

**Critical.** Remediated in supplied build.

### **Evidence**

server/socket/socketManager.js

### **Assessment**

A client could register an arbitrary user ID. Socket middleware now verifies JWT and binds identity server-side.

### **Tour-management impact**

For a tour platform, authorization failures can expose passenger records, modify itineraries, alter pricing, change booking state, assign guides/drivers/vehicles, manipulate payment state or reveal operational information. Controls therefore need both identity authorization and business-object authorization.

### **Acceptance criteria**

The control should be enforced server-side, default-deny, covered by an automated test, logged without sensitive payloads, and verified using both an allowed-role test and a denied-role/object test.



## **F-005 — remediation & verification**

### **Recommended design**

Implement and verify the control with a narrow allowlist and explicit business rules. Avoid relying on UI visibility. Add automated regression tests for anonymous, customer, agent, manager, guide, driver, admin and super-admin personas as appropriate. The supplied audit build already contains a targeted remediation for this finding; the change should still be exercised in integration tests before deployment.

### **Verification procedure**

1) Reproduce the original access path from an unauthenticated client where relevant. 2) Repeat with a least-privileged authenticated user. 3) Repeat with the intended privileged role. 4) Test an invalid object ID and another user's object ID. 5) Confirm the response does not disclose internal state. 6) Confirm an audit/security event exists where the action is security-sensitive.

### **Regression risk**

Security patches can break legacy frontend flows. Keep compatibility aliases only when their authorization policy is identical. Do not restore an insecure route merely to satisfy an old client; update the client instead.

### **Operational control**

Add the check to CI and to the release gate. A High finding should not be reopened without a documented compensating control and owner.

## **F-006 — Socket rooms accepted arbitrary room names**

### **Severity and status**

**High.** Remediated in supplied build.

### **Evidence**

server/socket/socketManager.js

### **Assessment**

An authenticated or unauthenticated client could attempt arbitrary room joins. Supplied build restricts joins to the caller's private user room.

### **Tour-management impact**

For a tour platform, authorization failures can expose passenger records, modify itineraries, alter pricing, change booking state, assign guides/drivers/vehicles, manipulate payment state or reveal operational information. Controls therefore need both identity authorization and business-object authorization.

### **Acceptance criteria**

The control should be enforced server-side, default-deny, covered by an automated test, logged without sensitive payloads, and verified using both an allowed-role test and a denied-role/object test.

## **F-006 — remediation & verification**

### **Recommended design**

Implement and verify the control with a narrow allowlist and explicit business rules. Avoid relying on UI visibility. Add automated regression tests for anonymous, customer, agent, manager, guide, driver, admin and super-admin personas as appropriate. The supplied audit build already contains a targeted remediation for this finding; the change should still be exercised in integration tests before deployment.

### **Verification procedure**

1) Reproduce the original access path from an unauthenticated client where relevant. 2) Repeat with a least-privileged authenticated user. 3) Repeat with the intended privileged role. 4) Test an invalid object ID and another user's object ID. 5) Confirm the response does not disclose internal state. 6) Confirm an audit/security event exists where the action is security-sensitive.

### **Regression risk**

Security patches can break legacy frontend flows. Keep compatibility aliases only when their authorization policy is identical. Do not restore an insecure route merely to satisfy an old client; update the client instead.

### **Operational control**

Add the check to CI and to the release gate. A High finding should not be reopened without a documented compensating control and owner.

# F-007 — JWT issuer/audience not enforced on verification

## Severity and status

**High.** Remediated in supplied build.

## Evidence

server/middleware/authMiddleware.js

## Assessment

Tokens were signed with issuer/audience but verification only supplied the secret. Verification now requires both claims.

## Tour-management impact

For a tour platform, authorization failures can expose passenger records, modify itineraries, alter pricing, change booking state, assign guides/drivers/vehicles, manipulate payment state or reveal operational information. Controls therefore need both identity authorization and business-object authorization.

## Acceptance criteria

The control should be enforced server-side, default-deny, covered by an automated test, logged without sensitive payloads, and verified using both an allowed-role test and a denied-role/object test.

## **F-007 — remediation & verification**

### **Recommended design**

Implement and verify the control with a narrow allowlist and explicit business rules. Avoid relying on UI visibility. Add automated regression tests for anonymous, customer, agent, manager, guide, driver, admin and super-admin personas as appropriate. The supplied audit build already contains a targeted remediation for this finding; the change should still be exercised in integration tests before deployment.

### **Verification procedure**

1) Reproduce the original access path from an unauthenticated client where relevant. 2) Repeat with a least-privileged authenticated user. 3) Repeat with the intended privileged role. 4) Test an invalid object ID and another user's object ID. 5) Confirm the response does not disclose internal state. 6) Confirm an audit/security event exists where the action is security-sensitive.

### **Regression risk**

Security patches can break legacy frontend flows. Keep compatibility aliases only when their authorization policy is identical. Do not restore an insecure route merely to satisfy an old client; update the client instead.

### **Operational control**

Add the check to CI and to the release gate. A High finding should not be reopened without a documented compensating control and owner.

# F-008 — Large global request body limits

## Severity and status

**High.** Remediated in supplied build.

## Evidence

server/app.js

## Assessment

10 MB JSON/urlencoded limits create unnecessary memory/CPU exposure. Supplied build uses 1 MB defaults; file uploads remain separately limited.

## Tour-management impact

For a tour platform, authorization failures can expose passenger records, modify itineraries, alter pricing, change booking state, assign guides/drivers/vehicles, manipulate payment state or reveal operational information. Controls therefore need both identity authorization and business-object authorization.

## Acceptance criteria

The control should be enforced server-side, default-deny, covered by an automated test, logged without sensitive payloads, and verified using both an allowed-role test and a denied-role/object test.

## **F-008 — remediation & verification**

### **Recommended design**

Implement and verify the control with a narrow allowlist and explicit business rules. Avoid relying on UI visibility. Add automated regression tests for anonymous, customer, agent, manager, guide, driver, admin and super-admin personas as appropriate. The supplied audit build already contains a targeted remediation for this finding; the change should still be exercised in integration tests before deployment.

### **Verification procedure**

1) Reproduce the original access path from an unauthenticated client where relevant. 2) Repeat with a least-privileged authenticated user. 3) Repeat with the intended privileged role. 4) Test an invalid object ID and another user's object ID. 5) Confirm the response does not disclose internal state. 6) Confirm an audit/security event exists where the action is security-sensitive.

### **Regression risk**

Security patches can break legacy frontend flows. Keep compatibility aliases only when their authorization policy is identical. Do not restore an insecure route merely to satisfy an old client; update the client instead.

### **Operational control**

Add the check to CI and to the release gate. A High finding should not be reopened without a documented compensating control and owner.

# **F-009 — Authentication endpoints lacked dedicated rate limiting**

## **Severity and status**

**High.** Remediated in supplied build.

## **Evidence**

server/routes/authRoutes.js; server/routes/adminAuthRoutes.js

## **Assessment**

Credential attacks and reset abuse need tighter controls than a generic limiter. Supplied build adds 10/15m login and 5/15m password-reset limits.

## **Tour-management impact**

For a tour platform, authorization failures can expose passenger records, modify itineraries, alter pricing, change booking state, assign guides/drivers/vehicles, manipulate payment state or reveal operational information. Controls therefore need both identity authorization and business-object authorization.

## **Acceptance criteria**

The control should be enforced server-side, default-deny, covered by an automated test, logged without sensitive payloads, and verified using both an allowed-role test and a denied-role/object test.



## **F-009 — remediation & verification**

### **Recommended design**

Implement and verify the control with a narrow allowlist and explicit business rules. Avoid relying on UI visibility. Add automated regression tests for anonymous, customer, agent, manager, guide, driver, admin and super-admin personas as appropriate. The supplied audit build already contains a targeted remediation for this finding; the change should still be exercised in integration tests before deployment.

### **Verification procedure**

1) Reproduce the original access path from an unauthenticated client where relevant. 2) Repeat with a least-privileged authenticated user. 3) Repeat with the intended privileged role. 4) Test an invalid object ID and another user's object ID. 5) Confirm the response does not disclose internal state. 6) Confirm an audit/security event exists where the action is security-sensitive.

### **Regression risk**

Security patches can break legacy frontend flows. Keep compatibility aliases only when their authorization policy is identical. Do not restore an insecure route merely to satisfy an old client; update the client instead.

### **Operational control**

Add the check to CI and to the release gate. A High finding should not be reopened without a documented compensating control and owner.

# F-010 — Multipart parser nesting/field limits incomplete

## Severity and status

**High.** Remediated in supplied build.

## Evidence

server/middleware/uploadMiddleware.js

## Assessment

Multer 2.2.0 is patched for 2026 DoS issues, but application-level multipart field/nesting limits are still required. Added explicit limits.

## Tour-management impact

For a tour platform, authorization failures can expose passenger records, modify itineraries, alter pricing, change booking state, assign guides/drivers/vehicles, manipulate payment state or reveal operational information. Controls therefore need both identity authorization and business-object authorization.

## Acceptance criteria

The control should be enforced server-side, default-deny, covered by an automated test, logged without sensitive payloads, and verified using both an allowed-role test and a denied-role/object test.

# F-010 — remediation & verification

## Recommended design

Implement and verify the control with a narrow allowlist and explicit business rules. Avoid relying on UI visibility. Add automated regression tests for anonymous, customer, agent, manager, guide, driver, admin and super-admin personas as appropriate. The supplied audit build already contains a targeted remediation for this finding; the change should still be exercised in integration tests before deployment.

## Verification procedure

1) Reproduce the original access path from an unauthenticated client where relevant. 2) Repeat with a least-privileged authenticated user. 3) Repeat with the intended privileged role. 4) Test an invalid object ID and another user's object ID. 5) Confirm the response does not disclose internal state. 6) Confirm an audit/security event exists where the action is security-sensitive.

## Regression risk

Security patches can break legacy frontend flows. Keep compatibility aliases only when their authorization policy is identical. Do not restore an insecure route merely to satisfy an old client; update the client instead.

## Operational control

Add the check to CI and to the release gate. A High finding should not be reopened without a documented compensating control and owner.

# F-011 — JWT stored in browser localStorage

## Severity and status

High. Open.

## Evidence

client/src/context/AuthContext.jsx; client/src/api/axios.js

## Assessment

Any successful XSS can read bearer tokens and impersonate users until expiry. Move toward httpOnly Secure SameSite cookies and short-lived access tokens.

## Tour-management impact

For a tour platform, authorization failures can expose passenger records, modify itineraries, alter pricing, change booking state, assign guides/drivers/vehicles, manipulate payment state or reveal operational information. Controls therefore need both identity authorization and business-object authorization.

## Acceptance criteria

The control should be enforced server-side, default-deny, covered by an automated test, logged without sensitive payloads, and verified using both an allowed-role test and a denied-role/object test.

## **F-011 — remediation & verification**

### **Recommended design**

Implement and verify the control with a narrow allowlist and explicit business rules. Avoid relying on UI visibility. Add automated regression tests for anonymous, customer, agent, manager, guide, driver, admin and super-admin personas as appropriate.

### **Verification procedure**

1) Reproduce the original access path from an unauthenticated client where relevant. 2) Repeat with a least-privileged authenticated user. 3) Repeat with the intended privileged role. 4) Test an invalid object ID and another user's object ID. 5) Confirm the response does not disclose internal state. 6) Confirm an audit/security event exists where the action is security-sensitive.

### **Regression risk**

Security patches can break legacy frontend flows. Keep compatibility aliases only when their authorization policy is identical. Do not restore an insecure route merely to satisfy an old client; update the client instead.

### **Operational control**

Add the check to CI and to the release gate. A High finding should not be reopened without a documented compensating control and owner.

# F-012 — Mass-assignment patterns in privileged controllers

## Severity and status

High. Open.

## Evidence

server/controllers/staffController.js; adminCouponController.js; tourManagerController.js; itineraryController.js; vehicleController.js; aiBookingController.js

## Assessment

req.body is passed wholesale or spread into trusted models. This risks privilege/state manipulation and business-rule bypass.

## Tour-management impact

For a tour platform, authorization failures can expose passenger records, modify itineraries, alter pricing, change booking state, assign guides/drivers/vehicles, manipulate payment state or reveal operational information. Controls therefore need both identity authorization and business-object authorization.

## Acceptance criteria

The control should be enforced server-side, default-deny, covered by an automated test, logged without sensitive payloads, and verified using both an allowed-role test and a denied-role/object test.

## **F-012 — remediation & verification**

### **Recommended design**

Implement and verify the control with a narrow allowlist and explicit business rules. Avoid relying on UI visibility. Add automated regression tests for anonymous, customer, agent, manager, guide, driver, admin and super-admin personas as appropriate.

### **Verification procedure**

1) Reproduce the original access path from an unauthenticated client where relevant. 2) Repeat with a least-privileged authenticated user. 3) Repeat with the intended privileged role. 4) Test an invalid object ID and another user's object ID. 5) Confirm the response does not disclose internal state. 6) Confirm an audit/security event exists where the action is security-sensitive.

### **Regression risk**

Security patches can break legacy frontend flows. Keep compatibility aliases only when their authorization policy is identical. Do not restore an insecure route merely to satisfy an old client; update the client instead.

### **Operational control**

Add the check to CI and to the release gate. A High finding should not be reopened without a documented compensating control and owner.

## **F-013 — Passport and health/emergency data stored as ordinary fields**

### **Severity and status**

**High.** Open.

### **Evidence**

server/models/Booking.js

### **Assessment**

Passport number, medical conditions, DOB and emergency contacts are high-risk personal/sensitive data. Apply minimization, field-level encryption/tokenization, strict projections, retention and access logging.

### **Tour-management impact**

For a tour platform, authorization failures can expose passenger records, modify itineraries, alter pricing, change booking state, assign guides/drivers/vehicles, manipulate payment state or reveal operational information. Controls therefore need both identity authorization and business-object authorization.

### **Acceptance criteria**

The control should be enforced server-side, default-deny, covered by an automated test, logged without sensitive payloads, and verified using both an allowed-role test and a denied-role/object test.



## **F-013 — remediation & verification**

### **Recommended design**

Implement and verify the control with a narrow allowlist and explicit business rules. Avoid relying on UI visibility. Add automated regression tests for anonymous, customer, agent, manager, guide, driver, admin and super-admin personas as appropriate.

### **Verification procedure**

1) Reproduce the original access path from an unauthenticated client where relevant. 2) Repeat with a least-privileged authenticated user. 3) Repeat with the intended privileged role. 4) Test an invalid object ID and another user's object ID. 5) Confirm the response does not disclose internal state. 6) Confirm an audit/security event exists where the action is security-sensitive.

### **Regression risk**

Security patches can break legacy frontend flows. Keep compatibility aliases only when their authorization policy is identical. Do not restore an insecure route merely to satisfy an old client; update the client instead.

### **Operational control**

Add the check to CI and to the release gate. A High finding should not be reopened without a documented compensating control and owner.

# **F-014 — Password reset code protection needs stronger defense**

## **Severity and status**

**High.** Partially remediated.

## **Evidence**

server/controllers/authController.js

## **Assessment**

Codes are SHA-256 hashed and expire in 10 minutes, but application-wide throttling, replay resistance, HMAC/pepper and atomic attempt increments should be strengthened.

## **Tour-management impact**

For a tour platform, authorization failures can expose passenger records, modify itineraries, alter pricing, change booking state, assign guides/drivers/vehicles, manipulate payment state or reveal operational information. Controls therefore need both identity authorization and business-object authorization.

## **Acceptance criteria**

The control should be enforced server-side, default-deny, covered by an automated test, logged without sensitive payloads, and verified using both an allowed-role test and a denied-role/object test.

## **F-014 — remediation & verification**

### **Recommended design**

Implement and verify the control with a narrow allowlist and explicit business rules. Avoid relying on UI visibility. Add automated regression tests for anonymous, customer, agent, manager, guide, driver, admin and super-admin personas as appropriate.

### **Verification procedure**

1) Reproduce the original access path from an unauthenticated client where relevant. 2) Repeat with a least-privileged authenticated user. 3) Repeat with the intended privileged role. 4) Test an invalid object ID and another user's object ID. 5) Confirm the response does not disclose internal state. 6) Confirm an audit/security event exists where the action is security-sensitive.

### **Regression risk**

Security patches can break legacy frontend flows. Keep compatibility aliases only when their authorization policy is identical. Do not restore an insecure route merely to satisfy an old client; update the client instead.

### **Operational control**

Add the check to CI and to the release gate. A High finding should not be reopened without a documented compensating control and owner.

## **F-015 — Registration immediately marks users verified**

### **Severity and status**

High. Open.

### **Evidence**

server/controllers/authController.js

### **Assessment**

isVerified is set true during registration. This undermines email/phone ownership assurance and can increase account abuse.

### **Tour-management impact**

For a tour platform, authorization failures can expose passenger records, modify itineraries, alter pricing, change booking state, assign guides/drivers/vehicles, manipulate payment state or reveal operational information. Controls therefore need both identity authorization and business-object authorization.

### **Acceptance criteria**

The control should be enforced server-side, default-deny, covered by an automated test, logged without sensitive payloads, and verified using both an allowed-role test and a denied-role/object test.

## **F-015 — remediation & verification**

### **Recommended design**

Implement and verify the control with a narrow allowlist and explicit business rules. Avoid relying on UI visibility. Add automated regression tests for anonymous, customer, agent, manager, guide, driver, admin and super-admin personas as appropriate.

### **Verification procedure**

1) Reproduce the original access path from an unauthenticated client where relevant. 2) Repeat with a least-privileged authenticated user. 3) Repeat with the intended privileged role. 4) Test an invalid object ID and another user's object ID. 5) Confirm the response does not disclose internal state. 6) Confirm an audit/security event exists where the action is security-sensitive.

### **Regression risk**

Security patches can break legacy frontend flows. Keep compatibility aliases only when their authorization policy is identical. Do not restore an insecure route merely to satisfy an old client; update the client instead.

### **Operational control**

Add the check to CI and to the release gate. A High finding should not be reopened without a documented compensating control and owner.

## **F-016 — Production CORS depends on optional environment variable**

### **Severity and status**

**High.** Open.

### **Evidence**

server/app.js; server/config/env.js

### **Assessment**

Fallback to localhost is acceptable for development but production should fail closed when allowed origins are missing or malformed.

### **Tour-management impact**

For a tour platform, authorization failures can expose passenger records, modify itineraries, alter pricing, change booking state, assign guides/drivers/vehicles, manipulate payment state or reveal operational information. Controls therefore need both identity authorization and business-object authorization.

### **Acceptance criteria**

The control should be enforced server-side, default-deny, covered by an automated test, logged without sensitive payloads, and verified using both an allowed-role test and a denied-role/object test.

## **F-016 — remediation & verification**

### **Recommended design**

Implement and verify the control with a narrow allowlist and explicit business rules. Avoid relying on UI visibility. Add automated regression tests for anonymous, customer, agent, manager, guide, driver, admin and super-admin personas as appropriate.

### **Verification procedure**

1) Reproduce the original access path from an unauthenticated client where relevant. 2) Repeat with a least-privileged authenticated user. 3) Repeat with the intended privileged role. 4) Test an invalid object ID and another user's object ID. 5) Confirm the response does not disclose internal state. 6) Confirm an audit/security event exists where the action is security-sensitive.

### **Regression risk**

Security patches can break legacy frontend flows. Keep compatibility aliases only when their authorization policy is identical. Do not restore an insecure route merely to satisfy an old client; update the client instead.

### **Operational control**

Add the check to CI and to the release gate. A High finding should not be reopened without a documented compensating control and owner.

## **F-017 — Legacy/duplicate role names increase authorization complexity**

### **Severity and status**

**Medium.** Open.

### **Evidence**

server/models/User.js; middleware files

### **Assessment**

admin/superadmin/administrator and multiple manager/guide aliases expand the authorization state space. Normalize roles in a single RBAC authority.

### **Tour-management impact**

For a tour platform, authorization failures can expose passenger records, modify itineraries, alter pricing, change booking state, assign guides/drivers/vehicles, manipulate payment state or reveal operational information. Controls therefore need both identity authorization and business-object authorization.

### **Acceptance criteria**

The control should be enforced server-side, default-deny, covered by an automated test, logged without sensitive payloads, and verified using both an allowed-role test and a denied-role/object test.



## **F-017 — remediation & verification**

### **Recommended design**

Implement and verify the control with a narrow allowlist and explicit business rules. Avoid relying on UI visibility. Add automated regression tests for anonymous, customer, agent, manager, guide, driver, admin and super-admin personas as appropriate.

### **Verification procedure**

1) Reproduce the original access path from an unauthenticated client where relevant. 2) Repeat with a least-privileged authenticated user. 3) Repeat with the intended privileged role. 4) Test an invalid object ID and another user's object ID. 5) Confirm the response does not disclose internal state. 6) Confirm an audit/security event exists where the action is security-sensitive.

### **Regression risk**

Security patches can break legacy frontend flows. Keep compatibility aliases only when their authorization policy is identical. Do not restore an insecure route merely to satisfy an old client; update the client instead.

### **Operational control**

Add the check to CI and to the release gate. A High finding should not be reopened without a documented compensating control and owner.

# **F-018 — Authorization is duplicated across multiple middleware implementations**

## **Severity and status**

**High.** Open.

## **Evidence**

server/middleware/authMiddleware.js; roleMiddleware.js; permissionMiddleware.js; adminMiddleware.js

## **Assessment**

Different paths use different permission semantics and legacy bypasses. Consolidate policy evaluation and default-deny.

## **Tour-management impact**

For a tour platform, authorization failures can expose passenger records, modify itineraries, alter pricing, change booking state, assign guides/drivers/vehicles, manipulate payment state or reveal operational information. Controls therefore need both identity authorization and business-object authorization.

## **Acceptance criteria**

The control should be enforced server-side, default-deny, covered by an automated test, logged without sensitive payloads, and verified using both an allowed-role test and a denied-role/object test.

# **F-018 — remediation & verification**

## **Recommended design**

Implement and verify the control with a narrow allowlist and explicit business rules. Avoid relying on UI visibility. Add automated regression tests for anonymous, customer, agent, manager, guide, driver, admin and super-admin personas as appropriate.

## **Verification procedure**

1) Reproduce the original access path from an unauthenticated client where relevant. 2) Repeat with a least-privileged authenticated user. 3) Repeat with the intended privileged role. 4) Test an invalid object ID and another user's object ID. 5) Confirm the response does not disclose internal state. 6) Confirm an audit/security event exists where the action is security-sensitive.

## **Regression risk**

Security patches can break legacy frontend flows. Keep compatibility aliases only when their authorization policy is identical. Do not restore an insecure route merely to satisfy an old client; update the client instead.

## **Operational control**

Add the check to CI and to the release gate. A High finding should not be reopened without a documented compensating control and owner.

# **F-019 — Client-side permission state is trusted for navigation**

## **Severity and status**

**High.** Open; defense-in-depth.

## **Evidence**

client/src/components/admin/ProtectedAdminRoute.jsx

## **Assessment**

UI guards are not security boundaries. The server is authoritative, but client state can be manipulated to reveal screens and trigger confusing flows.

## **Tour-management impact**

For a tour platform, authorization failures can expose passenger records, modify itineraries, alter pricing, change booking state, assign guides/drivers/vehicles, manipulate payment state or reveal operational information. Controls therefore need both identity authorization and business-object authorization.

## **Acceptance criteria**

The control should be enforced server-side, default-deny, covered by an automated test, logged without sensitive payloads, and verified using both an allowed-role test and a denied-role/object test.

## **F-019 — remediation & verification**

### **Recommended design**

Implement and verify the control with a narrow allowlist and explicit business rules. Avoid relying on UI visibility. Add automated regression tests for anonymous, customer, agent, manager, guide, driver, admin and super-admin personas as appropriate.

### **Verification procedure**

1) Reproduce the original access path from an unauthenticated client where relevant. 2) Repeat with a least-privileged authenticated user. 3) Repeat with the intended privileged role. 4) Test an invalid object ID and another user's object ID. 5) Confirm the response does not disclose internal state. 6) Confirm an audit/security event exists where the action is security-sensitive.

### **Regression risk**

Security patches can break legacy frontend flows. Keep compatibility aliases only when their authorization policy is identical. Do not restore an insecure route merely to satisfy an old client; update the client instead.

### **Operational control**

Add the check to CI and to the release gate. A High finding should not be reopened without a documented compensating control and owner.

## **F-020 — Verbose security/debug logging remains in several paths**

### **Severity and status**

**Medium.** Open.

### **Evidence**

server/controllers/\*; middleware/\*

### **Assessment**

Logs include emails, role data and potentially operational details. Production logs should use structured redaction and correlation IDs.

### **Tour-management impact**

For a tour platform, authorization failures can expose passenger records, modify itineraries, alter pricing, change booking state, assign guides/drivers/vehicles, manipulate payment state or reveal operational information. Controls therefore need both identity authorization and business-object authorization.

### **Acceptance criteria**

The control should be enforced server-side, default-deny, covered by an automated test, logged without sensitive payloads, and verified using both an allowed-role test and a denied-role/object test.

## **F-020 — remediation & verification**

### **Recommended design**

Implement and verify the control with a narrow allowlist and explicit business rules. Avoid relying on UI visibility. Add automated regression tests for anonymous, customer, agent, manager, guide, driver, admin and super-admin personas as appropriate.

### **Verification procedure**

1) Reproduce the original access path from an unauthenticated client where relevant. 2) Repeat with a least-privileged authenticated user. 3) Repeat with the intended privileged role. 4) Test an invalid object ID and another user's object ID. 5) Confirm the response does not disclose internal state. 6) Confirm an audit/security event exists where the action is security-sensitive.

### **Regression risk**

Security patches can break legacy frontend flows. Keep compatibility aliases only when their authorization policy is identical. Do not restore an insecure route merely to satisfy an old client; update the client instead.

### **Operational control**

Add the check to CI and to the release gate. A High finding should not be reopened without a documented compensating control and owner.

## **F-021 — No explicit CSRF strategy for cookie authentication**

### **Severity and status**

**High.** Open.

### **Evidence**

server/app.js; authMiddleware.js

### **Assessment**

withCredentials and cookie token support create CSRF exposure if cookie authentication is used. If cookies become primary, use SameSite plus CSRF tokens for state-changing requests.

### **Tour-management impact**

For a tour platform, authorization failures can expose passenger records, modify itineraries, alter pricing, change booking state, assign guides/drivers/vehicles, manipulate payment state or reveal operational information. Controls therefore need both identity authorization and business-object authorization.

### **Acceptance criteria**

The control should be enforced server-side, default-deny, covered by an automated test, logged without sensitive payloads, and verified using both an allowed-role test and a denied-role/object test.



## **F-021 — remediation & verification**

### **Recommended design**

Implement and verify the control with a narrow allowlist and explicit business rules. Avoid relying on UI visibility. Add automated regression tests for anonymous, customer, agent, manager, guide, driver, admin and super-admin personas as appropriate.

### **Verification procedure**

1) Reproduce the original access path from an unauthenticated client where relevant. 2) Repeat with a least-privileged authenticated user. 3) Repeat with the intended privileged role. 4) Test an invalid object ID and another user's object ID. 5) Confirm the response does not disclose internal state. 6) Confirm an audit/security event exists where the action is security-sensitive.

### **Regression risk**

Security patches can break legacy frontend flows. Keep compatibility aliases only when their authorization policy is identical. Do not restore an insecure route merely to satisfy an old client; update the client instead.

### **Operational control**

Add the check to CI and to the release gate. A High finding should not be reopened without a documented compensating control and owner.

## **F-022 — Booking capacity changes need atomic concurrency controls**

### **Severity and status**

**High.** Open.

### **Evidence**

server/models/Tour.js

### **Assessment**

bookSlot/releaseSlot are read-modify-save operations. Concurrent bookings can oversell capacity without atomic conditional updates/transactions.

### **Tour-management impact**

For a tour platform, authorization failures can expose passenger records, modify itineraries, alter pricing, change booking state, assign guides/drivers/vehicles, manipulate payment state or reveal operational information. Controls therefore need both identity authorization and business-object authorization.

### **Acceptance criteria**

The control should be enforced server-side, default-deny, covered by an automated test, logged without sensitive payloads, and verified using both an allowed-role test and a denied-role/object test.

## **F-022 — remediation & verification**

### **Recommended design**

Implement and verify the control with a narrow allowlist and explicit business rules. Avoid relying on UI visibility. Add automated regression tests for anonymous, customer, agent, manager, guide, driver, admin and super-admin personas as appropriate.

### **Verification procedure**

1) Reproduce the original access path from an unauthenticated client where relevant. 2) Repeat with a least-privileged authenticated user. 3) Repeat with the intended privileged role. 4) Test an invalid object ID and another user's object ID. 5) Confirm the response does not disclose internal state. 6) Confirm an audit/security event exists where the action is security-sensitive.

### **Regression risk**

Security patches can break legacy frontend flows. Keep compatibility aliases only when their authorization policy is identical. Do not restore an insecure route merely to satisfy an old client; update the client instead.

### **Operational control**

Add the check to CI and to the release gate. A High finding should not be reopened without a documented compensating control and owner.

## **F-023 — Payment state transitions require server-side invariants**

### **Severity and status**

**High.** Open.

### **Evidence**

server/models/Booking.js; payment controllers

### **Assessment**

Payment status, deposits, balances, refunds and commissions are financial state. All transitions must be idempotent, audited, authorization-checked and transactionally consistent.

### **Tour-management impact**

For a tour platform, authorization failures can expose passenger records, modify itineraries, alter pricing, change booking state, assign guides/drivers/vehicles, manipulate payment state or reveal operational information. Controls therefore need both identity authorization and business-object authorization.

### **Acceptance criteria**

The control should be enforced server-side, default-deny, covered by an automated test, logged without sensitive payloads, and verified using both an allowed-role test and a denied-role/object test.

## **F-023 — remediation & verification**

### **Recommended design**

Implement and verify the control with a narrow allowlist and explicit business rules. Avoid relying on UI visibility. Add automated regression tests for anonymous, customer, agent, manager, guide, driver, admin and super-admin personas as appropriate.

### **Verification procedure**

1) Reproduce the original access path from an unauthenticated client where relevant. 2) Repeat with a least-privileged authenticated user. 3) Repeat with the intended privileged role. 4) Test an invalid object ID and another user's object ID. 5) Confirm the response does not disclose internal state. 6) Confirm an audit/security event exists where the action is security-sensitive.

### **Regression risk**

Security patches can break legacy frontend flows. Keep compatibility aliases only when their authorization policy is identical. Do not restore an insecure route merely to satisfy an old client; update the client instead.

### **Operational control**

Add the check to CI and to the release gate. A High finding should not be reopened without a documented compensating control and owner.

## **F-024 — M-Pesa/webhook security must be independently verified**

### **Severity and status**

**High.** Open pending live provider validation.

### **Evidence**

server/routes/mpesaRoutes.js; controllers/mpesaController.js

### **Assessment**

Payment callbacks require provider signature/credential validation, replay protection, idempotency keys and amount/reference reconciliation before marking bookings paid.

### **Tour-management impact**

For a tour platform, authorization failures can expose passenger records, modify itineraries, alter pricing, change booking state, assign guides/drivers/vehicles, manipulate payment state or reveal operational information. Controls therefore need both identity authorization and business-object authorization.

### **Acceptance criteria**

The control should be enforced server-side, default-deny, covered by an automated test, logged without sensitive payloads, and verified using both an allowed-role test and a denied-role/object test.

## **F-024 — remediation & verification**

### **Recommended design**

Implement and verify the control with a narrow allowlist and explicit business rules. Avoid relying on UI visibility. Add automated regression tests for anonymous, customer, agent, manager, guide, driver, admin and super-admin personas as appropriate.

### **Verification procedure**

1) Reproduce the original access path from an unauthenticated client where relevant. 2) Repeat with a least-privileged authenticated user. 3) Repeat with the intended privileged role. 4) Test an invalid object ID and another user's object ID. 5) Confirm the response does not disclose internal state. 6) Confirm an audit/security event exists where the action is security-sensitive.

### **Regression risk**

Security patches can break legacy frontend flows. Keep compatibility aliases only when their authorization policy is identical. Do not restore an insecure route merely to satisfy an old client; update the client instead.

### **Operational control**

Add the check to CI and to the release gate. A High finding should not be reopened without a documented compensating control and owner.

## **F-025 — AI endpoints need data minimization and abuse controls**

### **Severity and status**

**High.** Open.

### **Evidence**

server/routes/aiRoutes.js; adminAIRoutes.js

### **Assessment**

Prompts may contain customer/booking information and can incur cost or leak sensitive data. Add role-specific limits, prompt logging redaction, model allowlists and outbound policy.

### **Tour-management impact**

For a tour platform, authorization failures can expose passenger records, modify itineraries, alter pricing, change booking state, assign guides/drivers/vehicles, manipulate payment state or reveal operational information. Controls therefore need both identity authorization and business-object authorization.

### **Acceptance criteria**

The control should be enforced server-side, default-deny, covered by an automated test, logged without sensitive payloads, and verified using both an allowed-role test and a denied-role/object test.



## **F-025 — remediation & verification**

### **Recommended design**

Implement and verify the control with a narrow allowlist and explicit business rules. Avoid relying on UI visibility. Add automated regression tests for anonymous, customer, agent, manager, guide, driver, admin and super-admin personas as appropriate.

### **Verification procedure**

1) Reproduce the original access path from an unauthenticated client where relevant. 2) Repeat with a least-privileged authenticated user. 3) Repeat with the intended privileged role. 4) Test an invalid object ID and another user's object ID. 5) Confirm the response does not disclose internal state. 6) Confirm an audit/security event exists where the action is security-sensitive.

### **Regression risk**

Security patches can break legacy frontend flows. Keep compatibility aliases only when their authorization policy is identical. Do not restore an insecure route merely to satisfy an old client; update the client instead.

### **Operational control**

Add the check to CI and to the release gate. A High finding should not be reopened without a documented compensating control and owner.

# Engineering domain review 01 — Architecture and modularity

## Assessment

The application has a broad monolithic Express API with many controllers/routes and a React SPA. This is viable, but route ownership and authorization become difficult as feature count grows. Introduce clear bounded modules—identity, tours, bookings, payments, operations, CRM, content, reporting, AI—and define service-layer contracts so controllers do not implement financial/business rules independently.

## Recommended target state

The target is a modular, testable service boundary with explicit authorization, validated inputs, transactional business invariants and observable failure modes. Security should be embedded in the domain layer rather than added only at route level.

## Definition of done

Document the invariant, implement it server-side, add negative tests, add monitoring where relevant, and verify the behavior in staging using realistic tour-management scenarios.

# Engineering domain review 02 — API contract quality

## Assessment

Use a consistent envelope, validation strategy, pagination contract, error taxonomy and idempotency model. Avoid returning raw Mongoose documents from privileged endpoints. Define OpenAPI schemas and validate request/response compatibility in CI.

## Recommended target state

The target is a modular, testable service boundary with explicit authorization, validated inputs, transactional business invariants and observable failure modes. Security should be embedded in the domain layer rather than added only at route level.

## Definition of done

Document the invariant, implement it server-side, add negative tests, add monitoring where relevant, and verify the behavior in staging using realistic tour-management scenarios.

# Engineering domain review 03 — Input validation

## Assessment

Mongoose provides useful persistence validation, but request validation should happen before business logic. Add Zod/Joi/Ajv or equivalent schemas per endpoint. Validate types, enums, ranges, dates, traveler counts, currency amounts, IDs and query pagination.

## Recommended target state

The target is a modular, testable service boundary with explicit authorization, validated inputs, transactional business invariants and observable failure modes. Security should be embedded in the domain layer rather than added only at route level.

## Definition of done

Document the invariant, implement it server-side, add negative tests, add monitoring where relevant, and verify the behavior in staging using realistic tour-management scenarios.

# Engineering domain review 04 — Authorization architecture

## Assessment

Choose one authoritative RBAC/ABAC policy engine. Keep role aliases only during a migration. Permission names should be constants, not scattered string literals. Object-level rules should be explicit: a customer can read only their bookings; an agent can manage only assigned/owned scope; a manager can manage operational resources within their domain.

## Recommended target state

The target is a modular, testable service boundary with explicit authorization, validated inputs, transactional business invariants and observable failure modes. Security should be embedded in the domain layer rather than added only at route level.

## Definition of done

Document the invariant, implement it server-side, add negative tests, add monitoring where relevant, and verify the behavior in staging using realistic tour-management scenarios.

# Engineering domain review 05 — Booking lifecycle

## Assessment

Define an explicit state machine. Legal transitions should be enumerated rather than inferred from arbitrary PUT bodies. Each transition should validate actor, current state, payment state, travel date, capacity, cancellation policy and audit requirements.

## Recommended target state

The target is a modular, testable service boundary with explicit authorization, validated inputs, transactional business invariants and observable failure modes. Security should be embedded in the domain layer rather than added only at route level.

## Definition of done

Document the invariant, implement it server-side, add negative tests, add monitoring where relevant, and verify the behavior in staging using realistic tour-management scenarios.

# Engineering domain review 06 — Capacity and concurrency

## Assessment

Capacity is a shared resource. Use atomic MongoDB updates with conditional predicates or a transaction. Example invariant:  $\text{bookedSlots} + \text{requestedSlots} \leq \text{totalSlots}$ . The operation must succeed once or fail without changing state. Avoid read-modify-save for high-contention booking paths.

## Recommended target state

The target is a modular, testable service boundary with explicit authorization, validated inputs, transactional business invariants and observable failure modes. Security should be embedded in the domain layer rather than added only at route level.

## Definition of done

Document the invariant, implement it server-side, add negative tests, add monitoring where relevant, and verify the behavior in staging using realistic tour-management scenarios.

# Engineering domain review 07 — Traveler data

## Assessment

Traveler profiles contain passport, DOB, emergency contact and health-related information. Separate operationally necessary fields from optional enrichment. Encrypt sensitive fields, restrict projections, avoid putting them into logs/emails/AI prompts, and define retention/deletion rules.

## Recommended target state

The target is a modular, testable service boundary with explicit authorization, validated inputs, transactional business invariants and observable failure modes. Security should be embedded in the domain layer rather than added only at route level.

## Definition of done

Document the invariant, implement it server-side, add negative tests, add monitoring where relevant, and verify the behavior in staging using realistic tour-management scenarios.



# Engineering domain review 08 — Payments

## Assessment

Payments must be treated as a ledger. Never trust the browser's amount, payment status or provider reference. Recalculate server-side, verify provider callbacks, make callbacks idempotent, reconcile amounts/currency/booking, and prevent duplicate refunds.

## Recommended target state

The target is a modular, testable service boundary with explicit authorization, validated inputs, transactional business invariants and observable failure modes. Security should be embedded in the domain layer rather than added only at route level.

## Definition of done

Document the invariant, implement it server-side, add negative tests, add monitoring where relevant, and verify the behavior in staging using realistic tour-management scenarios.

# Engineering domain review 09 — M-Pesa integration

## Assessment

Safaricom callbacks and payment APIs should be validated against provider documentation and current credentials. Store provider IDs, request timestamps and idempotency keys. A callback should move state only after authentication and reconciliation.

## Recommended target state

The target is a modular, testable service boundary with explicit authorization, validated inputs, transactional business invariants and observable failure modes. Security should be embedded in the domain layer rather than added only at route level.

## Definition of done

Document the invariant, implement it server-side, add negative tests, add monitoring where relevant, and verify the behavior in staging using realistic tour-management scenarios.

# Engineering domain review 10 — Stripe/card payments

## Assessment

If card payments are used, prefer hosted/tokenized provider flows and never store raw PAN/CVV. Keep PCI scope minimal. Verify webhook signatures and event IDs. Treat provider webhooks as asynchronous commands that must be replay-safe.

## Recommended target state

The target is a modular, testable service boundary with explicit authorization, validated inputs, transactional business invariants and observable failure modes. Security should be embedded in the domain layer rather than added only at route level.

## Definition of done

Document the invariant, implement it server-side, add negative tests, add monitoring where relevant, and verify the behavior in staging using realistic tour-management scenarios.

# Engineering domain review 11 — Invoices and documents

## Assessment

Invoice and ticket generation should authorize the underlying booking before serving a file. Use opaque identifiers, short-lived download URLs and content-disposition headers. Do not expose sequential IDs where they become an enumeration vector.

## Recommended target state

The target is a modular, testable service boundary with explicit authorization, validated inputs, transactional business invariants and observable failure modes. Security should be embedded in the domain layer rather than added only at route level.

## Definition of done

Document the invariant, implement it server-side, add negative tests, add monitoring where relevant, and verify the behavior in staging using realistic tour-management scenarios.

# Engineering domain review 12 — Uploads

## Assessment

Cloudinary is a useful boundary, but file security remains application responsibility. Validate extension and MIME, limit dimensions and count, quarantine risky formats, strip metadata where appropriate, and avoid serving uploaded content from an executable origin.

## Recommended target state

The target is a modular, testable service boundary with explicit authorization, validated inputs, transactional business invariants and observable failure modes. Security should be embedded in the domain layer rather than added only at route level.

## Definition of done

Document the invariant, implement it server-side, add negative tests, add monitoring where relevant, and verify the behavior in staging using realistic tour-management scenarios.

# Engineering domain review 13 — Notifications

## Assessment

Notifications should be authorized by recipient identity. Do not accept arbitrary recipient IDs from the browser. Queue notifications, deduplicate where necessary, and redact passport/health/payment information.

## Recommended target state

The target is a modular, testable service boundary with explicit authorization, validated inputs, transactional business invariants and observable failure modes. Security should be embedded in the domain layer rather than added only at route level.

## Definition of done

Document the invariant, implement it server-side, add negative tests, add monitoring where relevant, and verify the behavior in staging using realistic tour-management scenarios.

# Engineering domain review 14 — Socket architecture

## Assessment

Socket authentication must happen during handshake. The server should derive user identity from the verified token. Room names should be server-controlled or capability-based. Broadcasts must be role/tenant scoped.

## Recommended target state

The target is a modular, testable service boundary with explicit authorization, validated inputs, transactional business invariants and observable failure modes. Security should be embedded in the domain layer rather than added only at route level.

## Definition of done

Document the invariant, implement it server-side, add negative tests, add monitoring where relevant, and verify the behavior in staging using realistic tour-management scenarios.

# Engineering domain review 15 — AI security

## Assessment

AI endpoints should be treated as untrusted-input processors. Do not put secrets or unrestricted database context into prompts. Redact PII, define prompt/response limits, enforce role-specific tools, validate model outputs before executing actions, and log decision metadata rather than sensitive prompt contents.

## Recommended target state

The target is a modular, testable service boundary with explicit authorization, validated inputs, transactional business invariants and observable failure modes. Security should be embedded in the domain layer rather than added only at route level.

## Definition of done

Document the invariant, implement it server-side, add negative tests, add monitoring where relevant, and verify the behavior in staging using realistic tour-management scenarios.



# Engineering domain review 16 — CRM and customer management

## Assessment

CRM endpoints can expose broad PII. Use field-level authorization and pagination. Search endpoints should have rate limits and should not permit arbitrary regex patterns that cause expensive queries.

## Recommended target state

The target is a modular, testable service boundary with explicit authorization, validated inputs, transactional business invariants and observable failure modes. Security should be embedded in the domain layer rather than added only at route level.

## Definition of done

Document the invariant, implement it server-side, add negative tests, add monitoring where relevant, and verify the behavior in staging using realistic tour-management scenarios.

# Engineering domain review 17 — Reporting and exports

## Assessment

CSV/PDF exports are high-value data exfiltration paths. Apply the same authorization as the underlying records, limit date ranges, log exports, avoid formula injection in spreadsheets, and use asynchronous generation for large reports.

## Recommended target state

The target is a modular, testable service boundary with explicit authorization, validated inputs, transactional business invariants and observable failure modes. Security should be embedded in the domain layer rather than added only at route level.

## Definition of done

Document the invariant, implement it server-side, add negative tests, add monitoring where relevant, and verify the behavior in staging using realistic tour-management scenarios.

# Engineering domain review 18 — Caching and CDN

## Assessment

Do not cache personalized booking, payment, invoice or traveler responses publicly. Use Cache-Control: private/no-store for sensitive endpoints. Cache public tours/destinations with explicit invalidation.

## Recommended target state

The target is a modular, testable service boundary with explicit authorization, validated inputs, transactional business invariants and observable failure modes. Security should be embedded in the domain layer rather than added only at route level.

## Definition of done

Document the invariant, implement it server-side, add negative tests, add monitoring where relevant, and verify the behavior in staging using realistic tour-management scenarios.

# Engineering domain review 19 — Frontend security

## Assessment

React escapes normal text rendering, but dangerous HTML injection, third-party scripts and URL handling still require review. Use a strict Content Security Policy in production and avoid arbitrary `dangerouslySetInnerHTML`.

## Recommended target state

The target is a modular, testable service boundary with explicit authorization, validated inputs, transactional business invariants and observable failure modes. Security should be embedded in the domain layer rather than added only at route level.

## Definition of done

Document the invariant, implement it server-side, add negative tests, add monitoring where relevant, and verify the behavior in staging using realistic tour-management scenarios.

# Engineering domain review 20 — Authentication UX

## Assessment

Move away from localStorage bearer tokens where feasible. Use short-lived access tokens and refresh rotation with httpOnly Secure SameSite cookies. Add session/device management and global revocation for compromised accounts.

## Recommended target state

The target is a modular, testable service boundary with explicit authorization, validated inputs, transactional business invariants and observable failure modes. Security should be embedded in the domain layer rather than added only at route level.

## Definition of done

Document the invariant, implement it server-side, add negative tests, add monitoring where relevant, and verify the behavior in staging using realistic tour-management scenarios.

# Engineering domain review 21 — Password security

## Assessment

bcrypt is appropriate, but tune cost through benchmarking and add breached-password screening if policy permits. Enforce reset throttling, invalidate sessions after password change, and avoid logging authentication secrets.

## Recommended target state

The target is a modular, testable service boundary with explicit authorization, validated inputs, transactional business invariants and observable failure modes. Security should be embedded in the domain layer rather than added only at route level.

## Definition of done

Document the invariant, implement it server-side, add negative tests, add monitoring where relevant, and verify the behavior in staging using realistic tour-management scenarios.

# Engineering domain review 22 — Secrets management

## Assessment

JWT, database, Cloudinary, M-Pesa, email, OpenAI and payment credentials must never be committed. Production should use a secret manager or platform secrets. Rotate credentials and maintain a documented rotation procedure.

## Recommended target state

The target is a modular, testable service boundary with explicit authorization, validated inputs, transactional business invariants and observable failure modes. Security should be embedded in the domain layer rather than added only at route level.

## Definition of done

Document the invariant, implement it server-side, add negative tests, add monitoring where relevant, and verify the behavior in staging using realistic tour-management scenarios.

# Engineering domain review 23 — Database security

## Assessment

MongoDB should use TLS, authentication, least-privileged DB users, network allowlists/private networking and backups. Disable direct public exposure. Use indexes deliberately and monitor slow queries.

## Recommended target state

The target is a modular, testable service boundary with explicit authorization, validated inputs, transactional business invariants and observable failure modes. Security should be embedded in the domain layer rather than added only at route level.

## Definition of done

Document the invariant, implement it server-side, add negative tests, add monitoring where relevant, and verify the behavior in staging using realistic tour-management scenarios.



# Engineering domain review 24 — Observability

## Assessment

Add correlation IDs, structured JSON logs, security events, metrics and traces. Redact tokens, passwords, reset codes, medical data, passport numbers and payment secrets. Monitor authorization failures and unusual booking/payment activity.

## Recommended target state

The target is a modular, testable service boundary with explicit authorization, validated inputs, transactional business invariants and observable failure modes. Security should be embedded in the domain layer rather than added only at route level.

## Definition of done

Document the invariant, implement it server-side, add negative tests, add monitoring where relevant, and verify the behavior in staging using realistic tour-management scenarios.

# Engineering domain review 25 — Backup and disaster recovery

## Assessment

Backups are only useful if restores are tested. Define RPO/RTO, encrypted backups, immutable/offsite copies, restore drills and retention. Backup download endpoints require the strongest authorization.

## Recommended target state

The target is a modular, testable service boundary with explicit authorization, validated inputs, transactional business invariants and observable failure modes. Security should be embedded in the domain layer rather than added only at route level.

## Definition of done

Document the invariant, implement it server-side, add negative tests, add monitoring where relevant, and verify the behavior in staging using realistic tour-management scenarios.

# Engineering domain review 26 — CI/CD and supply chain

## Assessment

Use lockfiles, Dependabot/Renovate, npm audit signatures, SBOM generation, secret scanning, SAST, dependency scanning and protected deployment branches. Pin runtime versions and test production builds in CI.

## Recommended target state

The target is a modular, testable service boundary with explicit authorization, validated inputs, transactional business invariants and observable failure modes. Security should be embedded in the domain layer rather than added only at route level.

## Definition of done

Document the invariant, implement it server-side, add negative tests, add monitoring where relevant, and verify the behavior in staging using realistic tour-management scenarios.

# Engineering domain review 27 — Testing strategy

## Assessment

Unit tests protect pricing and state machines; integration tests protect authorization and payments; E2E tests protect customer booking flows. Security tests should deliberately attempt IDOR, privilege escalation, mass assignment, replay, rate-limit bypass and file-upload abuse.

## Recommended target state

The target is a modular, testable service boundary with explicit authorization, validated inputs, transactional business invariants and observable failure modes. Security should be embedded in the domain layer rather than added only at route level.

## Definition of done

Document the invariant, implement it server-side, add negative tests, add monitoring where relevant, and verify the behavior in staging using realistic tour-management scenarios.

# Engineering domain review 28 — Performance and scalability

## Assessment

Add indexes for high-cardinality booking/tour queries, pagination everywhere, query projections and bounded populations. Avoid N+1 populate patterns. Use queues for emails, PDFs, AI and exports.

## Recommended target state

The target is a modular, testable service boundary with explicit authorization, validated inputs, transactional business invariants and observable failure modes. Security should be embedded in the domain layer rather than added only at route level.

## Definition of done

Document the invariant, implement it server-side, add negative tests, add monitoring where relevant, and verify the behavior in staging using realistic tour-management scenarios.

# Engineering domain review 29 — Reliability

## Assessment

Use graceful shutdown, request timeouts, circuit breakers for third-party services, retry policies with jitter and idempotency. Do not retry non-idempotent payment operations blindly.

## Recommended target state

The target is a modular, testable service boundary with explicit authorization, validated inputs, transactional business invariants and observable failure modes. Security should be embedded in the domain layer rather than added only at route level.

## Definition of done

Document the invariant, implement it server-side, add negative tests, add monitoring where relevant, and verify the behavior in staging using realistic tour-management scenarios.

# Engineering domain review 30 — Privacy governance

## Assessment

The Kenya Data Protection Act requires lawful, fair and transparent processing, purpose limitation, minimization, storage limitation and safeguards. Map data flows, publish notices, support data-subject rights, assess cross-border transfers and maintain incident procedures.

## Recommended target state

The target is a modular, testable service boundary with explicit authorization, validated inputs, transactional business invariants and observable failure modes. Security should be embedded in the domain layer rather than added only at route level.

## Definition of done

Document the invariant, implement it server-side, add negative tests, add monitoring where relevant, and verify the behavior in staging using realistic tour-management scenarios.

# Engineering domain review 31 — Tour operations

## Assessment

Guide/driver/vehicle assignment needs conflict detection, availability windows and authorization. Prevent double assignment and ensure vehicle passenger capacity meets booking requirements.

## Recommended target state

The target is a modular, testable service boundary with explicit authorization, validated inputs, transactional business invariants and observable failure modes. Security should be embedded in the domain layer rather than added only at route level.

## Definition of done

Document the invariant, implement it server-side, add negative tests, add monitoring where relevant, and verify the behavior in staging using realistic tour-management scenarios.



# Engineering domain review 32 — Cancellation and refunds

## Assessment

Cancellation rules should be deterministic and versioned. Calculate refund eligibility server-side from booking date, travel date, policy version and payment ledger. Audit every refund decision.

## Recommended target state

The target is a modular, testable service boundary with explicit authorization, validated inputs, transactional business invariants and observable failure modes. Security should be embedded in the domain layer rather than added only at route level.

## Definition of done

Document the invariant, implement it server-side, add negative tests, add monitoring where relevant, and verify the behavior in staging using realistic tour-management scenarios.

# Engineering domain review 33 — Pricing integrity

## Assessment

Price must be calculated from trusted tour configuration, date, traveler count, discounts, tax, service fees and coupon rules. Never accept totalAmount from the client as authoritative.

## Recommended target state

The target is a modular, testable service boundary with explicit authorization, validated inputs, transactional business invariants and observable failure modes. Security should be embedded in the domain layer rather than added only at route level.

## Definition of done

Document the invariant, implement it server-side, add negative tests, add monitoring where relevant, and verify the behavior in staging using realistic tour-management scenarios.

# Engineering domain review 34 — Coupon security

## Assessment

Coupons need scope, expiry, usage limits, customer limits and atomic redemption. Prevent negative totals, stacked discounts unless explicitly allowed, and race-condition over-redemption.

## Recommended target state

The target is a modular, testable service boundary with explicit authorization, validated inputs, transactional business invariants and observable failure modes. Security should be embedded in the domain layer rather than added only at route level.

## Definition of done

Document the invariant, implement it server-side, add negative tests, add monitoring where relevant, and verify the behavior in staging using realistic tour-management scenarios.

# Engineering domain review 35 — Agent commissions

## Assessment

Commission rates and amounts should be server-owned. Approvals and payouts need segregation of duties and immutable ledger records.

## Recommended target state

The target is a modular, testable service boundary with explicit authorization, validated inputs, transactional business invariants and observable failure modes. Security should be embedded in the domain layer rather than added only at route level.

## Definition of done

Document the invariant, implement it server-side, add negative tests, add monitoring where relevant, and verify the behavior in staging using realistic tour-management scenarios.

# Engineering domain review 36 — Audit logging

## Assessment

Audit logs should record actor, action, resource, resource ID, outcome, timestamp, correlation ID and reason. Never allow ordinary users to alter audit records.

## Recommended target state

The target is a modular, testable service boundary with explicit authorization, validated inputs, transactional business invariants and observable failure modes. Security should be embedded in the domain layer rather than added only at route level.

## Definition of done

Document the invariant, implement it server-side, add negative tests, add monitoring where relevant, and verify the behavior in staging using realistic tour-management scenarios.

# Engineering domain review 37 — Incident response

## Assessment

Create playbooks for credential theft, data exposure, payment fraud, account takeover, ransomware, provider compromise and unauthorized staff actions. Include containment, evidence preservation, communication and recovery.

## Recommended target state

The target is a modular, testable service boundary with explicit authorization, validated inputs, transactional business invariants and observable failure modes. Security should be embedded in the domain layer rather than added only at route level.

## Definition of done

Document the invariant, implement it server-side, add negative tests, add monitoring where relevant, and verify the behavior in staging using realistic tour-management scenarios.

# Engineering domain review 38 — Business continuity

## Assessment

Tour operations depend on external providers and human staff. Define degraded modes: payment provider unavailable, SMS unavailable, Cloudinary unavailable, MongoDB failover, AI unavailable. Customer booking should remain safe without AI.

## Recommended target state

The target is a modular, testable service boundary with explicit authorization, validated inputs, transactional business invariants and observable failure modes. Security should be embedded in the domain layer rather than added only at route level.

## Definition of done

Document the invariant, implement it server-side, add negative tests, add monitoring where relevant, and verify the behavior in staging using realistic tour-management scenarios.

# Engineering domain review 39 — Accessibility and UX quality

## Assessment

WCAG-oriented keyboard navigation, labels, contrast, form errors and mobile usability are important for booking conversion and operational dashboards. Accessibility is also a reliability control: clear status and error messages reduce operational mistakes.

## Recommended target state

The target is a modular, testable service boundary with explicit authorization, validated inputs, transactional business invariants and observable failure modes. Security should be embedded in the domain layer rather than added only at route level.

## Definition of done

Document the invariant, implement it server-side, add negative tests, add monitoring where relevant, and verify the behavior in staging using realistic tour-management scenarios.



# Engineering domain review 40 — Internationalization

## Assessment

Date/time and currency must be explicit. Store canonical UTC timestamps and render in the booking locale/time zone. Avoid ambiguous date parsing.

## Recommended target state

The target is a modular, testable service boundary with explicit authorization, validated inputs, transactional business invariants and observable failure modes. Security should be embedded in the domain layer rather than added only at route level.

## Definition of done

Document the invariant, implement it server-side, add negative tests, add monitoring where relevant, and verify the behavior in staging using realistic tour-management scenarios.

# Engineering domain review 41 — SEO and public content

## Assessment

Sitemaps and public destination/tour pages should exclude unpublished/deleted records. Sanitize rich content and ensure canonical URLs. Do not leak draft itinerary or private operational notes.

## Recommended target state

The target is a modular, testable service boundary with explicit authorization, validated inputs, transactional business invariants and observable failure modes. Security should be embedded in the domain layer rather than added only at route level.

## Definition of done

Document the invariant, implement it server-side, add negative tests, add monitoring where relevant, and verify the behavior in staging using realistic tour-management scenarios.

# Engineering domain review 42 — Configuration management

## Assessment

Fail closed in production when critical settings are absent. Maintain a typed environment schema and separate development defaults from production requirements.

## Recommended target state

The target is a modular, testable service boundary with explicit authorization, validated inputs, transactional business invariants and observable failure modes. Security should be embedded in the domain layer rather than added only at route level.

## Definition of done

Document the invariant, implement it server-side, add negative tests, add monitoring where relevant, and verify the behavior in staging using realistic tour-management scenarios.

# Engineering domain review 43 — Release management

## Assessment

Use versioned migrations, change records, rollback plans and feature flags for high-risk changes. Never deploy database schema changes without compatibility planning.

## Recommended target state

The target is a modular, testable service boundary with explicit authorization, validated inputs, transactional business invariants and observable failure modes. Security should be embedded in the domain layer rather than added only at route level.

## Definition of done

Document the invariant, implement it server-side, add negative tests, add monitoring where relevant, and verify the behavior in staging using realistic tour-management scenarios.

# Engineering domain review 44 — Security ownership

## Assessment

Assign owners for IAM, payment security, privacy, infrastructure, backups and incident response. A technical control without an owner tends to decay.

## Recommended target state

The target is a modular, testable service boundary with explicit authorization, validated inputs, transactional business invariants and observable failure modes. Security should be embedded in the domain layer rather than added only at route level.

## Definition of done

Document the invariant, implement it server-side, add negative tests, add monitoring where relevant, and verify the behavior in staging using realistic tour-management scenarios.

# Engineering domain review 45 — Go-live gate

## Assessment

The release should require zero Critical findings, zero unaccepted High findings, successful security regression suite, verified payment webhook tests, restore test, secret scan and production configuration review.

## Recommended target state

The target is a modular, testable service boundary with explicit authorization, validated inputs, transactional business invariants and observable failure modes. Security should be embedded in the domain layer rather than added only at route level.

## Definition of done

Document the invariant, implement it server-side, add negative tests, add monitoring where relevant, and verify the behavior in staging using realistic tour-management scenarios.

# OWASP ASVS 5.0 mapping 01 — Architecture, threat modeling and trust boundaries

## Assessment

Use the ASVS requirement set as the detailed verification checklist. The application should demonstrate evidence rather than merely claim compliance. Map each control to a code location, test case and operational owner.

## Current posture

The supplied source demonstrates partial controls—Helmet, JWT, bcrypt, validation in some Mongoose models, RBAC middleware, rate limiting and upload limits—but coverage is inconsistent across the large route surface.

## Required evidence

For release, maintain a verification matrix containing requirement ID, implementation reference, automated test, manual test where required, result, evidence artifact and owner. Re-run it after major architectural changes.

# OWASP ASVS 5.0 mapping 02 — Authentication verification

## Assessment

Use the ASVS requirement set as the detailed verification checklist. The application should demonstrate evidence rather than merely claim compliance. Map each control to a code location, test case and operational owner.

## Current posture

The supplied source demonstrates partial controls—Helmet, JWT, bcrypt, validation in some Mongoose models, RBAC middleware, rate limiting and upload limits—but coverage is inconsistent across the large route surface.

## Required evidence

For release, maintain a verification matrix containing requirement ID, implementation reference, automated test, manual test where required, result, evidence artifact and owner. Re-run it after major architectural changes.



# OWASP ASVS 5.0 mapping 03 — Session management

## Assessment

Use the ASVS requirement set as the detailed verification checklist. The application should demonstrate evidence rather than merely claim compliance. Map each control to a code location, test case and operational owner.

## Current posture

The supplied source demonstrates partial controls—Helmet, JWT, bcrypt, validation in some Mongoose models, RBAC middleware, rate limiting and upload limits—but coverage is inconsistent across the large route surface.

## Required evidence

For release, maintain a verification matrix containing requirement ID, implementation reference, automated test, manual test where required, result, evidence artifact and owner. Re-run it after major architectural changes.

# OWASP ASVS 5.0 mapping 04 — Access control

## Assessment

Use the ASVS requirement set as the detailed verification checklist. The application should demonstrate evidence rather than merely claim compliance. Map each control to a code location, test case and operational owner.

## Current posture

The supplied source demonstrates partial controls—Helmet, JWT, bcrypt, validation in some Mongoose models, RBAC middleware, rate limiting and upload limits—but coverage is inconsistent across the large route surface.

## Required evidence

For release, maintain a verification matrix containing requirement ID, implementation reference, automated test, manual test where required, result, evidence artifact and owner. Re-run it after major architectural changes.

# OWASP ASVS 5.0 mapping 05 — Validation, sanitization and encoding

## Assessment

Use the ASVS requirement set as the detailed verification checklist. The application should demonstrate evidence rather than merely claim compliance. Map each control to a code location, test case and operational owner.

## Current posture

The supplied source demonstrates partial controls—Helmet, JWT, bcrypt, validation in some Mongoose models, RBAC middleware, rate limiting and upload limits—but coverage is inconsistent across the large route surface.

## Required evidence

For release, maintain a verification matrix containing requirement ID, implementation reference, automated test, manual test where required, result, evidence artifact and owner. Re-run it after major architectural changes.

# OWASP ASVS 5.0 mapping 06 — Cryptography and secrets

## Assessment

Use the ASVS requirement set as the detailed verification checklist. The application should demonstrate evidence rather than merely claim compliance. Map each control to a code location, test case and operational owner.

## Current posture

The supplied source demonstrates partial controls—Helmet, JWT, bcrypt, validation in some Mongoose models, RBAC middleware, rate limiting and upload limits—but coverage is inconsistent across the large route surface.

## Required evidence

For release, maintain a verification matrix containing requirement ID, implementation reference, automated test, manual test where required, result, evidence artifact and owner. Re-run it after major architectural changes.

# OWASP ASVS 5.0 mapping 07 — Error handling and logging

## Assessment

Use the ASVS requirement set as the detailed verification checklist. The application should demonstrate evidence rather than merely claim compliance. Map each control to a code location, test case and operational owner.

## Current posture

The supplied source demonstrates partial controls—Helmet, JWT, bcrypt, validation in some Mongoose models, RBAC middleware, rate limiting and upload limits—but coverage is inconsistent across the large route surface.

## Required evidence

For release, maintain a verification matrix containing requirement ID, implementation reference, automated test, manual test where required, result, evidence artifact and owner. Re-run it after major architectural changes.

# OWASP ASVS 5.0 mapping 08 — Data protection and privacy

## Assessment

Use the ASVS requirement set as the detailed verification checklist. The application should demonstrate evidence rather than merely claim compliance. Map each control to a code location, test case and operational owner.

## Current posture

The supplied source demonstrates partial controls—Helmet, JWT, bcrypt, validation in some Mongoose models, RBAC middleware, rate limiting and upload limits—but coverage is inconsistent across the large route surface.

## Required evidence

For release, maintain a verification matrix containing requirement ID, implementation reference, automated test, manual test where required, result, evidence artifact and owner. Re-run it after major architectural changes.

# OWASP ASVS 5.0 mapping 09 — API communications and web security

## Assessment

Use the ASVS requirement set as the detailed verification checklist. The application should demonstrate evidence rather than merely claim compliance. Map each control to a code location, test case and operational owner.

## Current posture

The supplied source demonstrates partial controls—Helmet, JWT, bcrypt, validation in some Mongoose models, RBAC middleware, rate limiting and upload limits—but coverage is inconsistent across the large route surface.

## Required evidence

For release, maintain a verification matrix containing requirement ID, implementation reference, automated test, manual test where required, result, evidence artifact and owner. Re-run it after major architectural changes.

# OWASP ASVS 5.0 mapping 10 — Configuration and dependency security

## Assessment

Use the ASVS requirement set as the detailed verification checklist. The application should demonstrate evidence rather than merely claim compliance. Map each control to a code location, test case and operational owner.

## Current posture

The supplied source demonstrates partial controls—Helmet, JWT, bcrypt, validation in some Mongoose models, RBAC middleware, rate limiting and upload limits—but coverage is inconsistent across the large route surface.

## Required evidence

For release, maintain a verification matrix containing requirement ID, implementation reference, automated test, manual test where required, result, evidence artifact and owner. Re-run it after major architectural changes.



# OWASP API Security Top 10 — API1 Broken Object Level Authorization

## Assessment

This API category is particularly relevant to a tour platform because IDs for bookings, tours, users, payments, invoices, vehicles and reports are routinely exposed to clients. Every endpoint must enforce both function-level and object-level authorization.

## Tour-management test scenario

Attempt the operation with a valid token belonging to a different customer/agent/manager. A secure implementation must deny the operation without revealing whether the target object exists, unless existence is intentionally public.

## Required evidence

Maintain negative integration tests for every sensitive resource and role. Test query filters, path IDs, body IDs and nested IDs because authorization failures can occur in any one of those locations.

# OWASP API Security Top 10 — API2 Broken Authentication

## Assessment

This API category is particularly relevant to a tour platform because IDs for bookings, tours, users, payments, invoices, vehicles and reports are routinely exposed to clients. Every endpoint must enforce both function-level and object-level authorization.

## Tour-management test scenario

Attempt the operation with a valid token belonging to a different customer/agent/manager. A secure implementation must deny the operation without revealing whether the target object exists, unless existence is intentionally public.

## Required evidence

Maintain negative integration tests for every sensitive resource and role. Test query filters, path IDs, body IDs and nested IDs because authorization failures can occur in any one of those locations.

# OWASP API Security Top 10 — API3 Broken Object Property Level Authorization

## Assessment

This API category is particularly relevant to a tour platform because IDs for bookings, tours, users, payments, invoices, vehicles and reports are routinely exposed to clients. Every endpoint must enforce both function-level and object-level authorization.

## Tour-management test scenario

Attempt the operation with a valid token belonging to a different customer/agent/manager. A secure implementation must deny the operation without revealing whether the target object exists, unless existence is intentionally public.

## Required evidence

Maintain negative integration tests for every sensitive resource and role. Test query filters, path IDs, body IDs and nested IDs because authorization failures can occur in any one of those locations.

# OWASP API Security Top 10 — API4 Unrestricted Resource Consumption

## Assessment

This API category is particularly relevant to a tour platform because IDs for bookings, tours, users, payments, invoices, vehicles and reports are routinely exposed to clients. Every endpoint must enforce both function-level and object-level authorization.

## Tour-management test scenario

Attempt the operation with a valid token belonging to a different customer/agent/manager. A secure implementation must deny the operation without revealing whether the target object exists, unless existence is intentionally public.

## Required evidence

Maintain negative integration tests for every sensitive resource and role. Test query filters, path IDs, body IDs and nested IDs because authorization failures can occur in any one of those locations.

# OWASP API Security Top 10 — API5 Broken Function Level Authorization

## Assessment

This API category is particularly relevant to a tour platform because IDs for bookings, tours, users, payments, invoices, vehicles and reports are routinely exposed to clients. Every endpoint must enforce both function-level and object-level authorization.

## Tour-management test scenario

Attempt the operation with a valid token belonging to a different customer/agent/manager. A secure implementation must deny the operation without revealing whether the target object exists, unless existence is intentionally public.

## Required evidence

Maintain negative integration tests for every sensitive resource and role. Test query filters, path IDs, body IDs and nested IDs because authorization failures can occur in any one of those locations.

# OWASP API Security Top 10 — API6 Sensitive Business Flows

## Assessment

This API category is particularly relevant to a tour platform because IDs for bookings, tours, users, payments, invoices, vehicles and reports are routinely exposed to clients. Every endpoint must enforce both function-level and object-level authorization.

## Tour-management test scenario

Attempt the operation with a valid token belonging to a different customer/agent/manager. A secure implementation must deny the operation without revealing whether the target object exists, unless existence is intentionally public.

## Required evidence

Maintain negative integration tests for every sensitive resource and role. Test query filters, path IDs, body IDs and nested IDs because authorization failures can occur in any one of those locations.

# OWASP API Security Top 10 — API7 Server-Side Request Forgery

## Assessment

This API category is particularly relevant to a tour platform because IDs for bookings, tours, users, payments, invoices, vehicles and reports are routinely exposed to clients. Every endpoint must enforce both function-level and object-level authorization.

## Tour-management test scenario

Attempt the operation with a valid token belonging to a different customer/agent/manager. A secure implementation must deny the operation without revealing whether the target object exists, unless existence is intentionally public.

## Required evidence

Maintain negative integration tests for every sensitive resource and role. Test query filters, path IDs, body IDs and nested IDs because authorization failures can occur in any one of those locations.

# OWASP API Security Top 10 — API8 Security Misconfiguration

## Assessment

This API category is particularly relevant to a tour platform because IDs for bookings, tours, users, payments, invoices, vehicles and reports are routinely exposed to clients. Every endpoint must enforce both function-level and object-level authorization.

## Tour-management test scenario

Attempt the operation with a valid token belonging to a different customer/agent/manager. A secure implementation must deny the operation without revealing whether the target object exists, unless existence is intentionally public.

## Required evidence

Maintain negative integration tests for every sensitive resource and role. Test query filters, path IDs, body IDs and nested IDs because authorization failures can occur in any one of those locations.



# OWASP API Security Top 10 — API9 Improper Inventory Management

## Assessment

This API category is particularly relevant to a tour platform because IDs for bookings, tours, users, payments, invoices, vehicles and reports are routinely exposed to clients. Every endpoint must enforce both function-level and object-level authorization.

## Tour-management test scenario

Attempt the operation with a valid token belonging to a different customer/agent/manager. A secure implementation must deny the operation without revealing whether the target object exists, unless existence is intentionally public.

## Required evidence

Maintain negative integration tests for every sensitive resource and role. Test query filters, path IDs, body IDs and nested IDs because authorization failures can occur in any one of those locations.

# OWASP API Security Top 10 — API10 Unsafe Consumption of APIs

## Assessment

This API category is particularly relevant to a tour platform because IDs for bookings, tours, users, payments, invoices, vehicles and reports are routinely exposed to clients. Every endpoint must enforce both function-level and object-level authorization.

## Tour-management test scenario

Attempt the operation with a valid token belonging to a different customer/agent/manager. A secure implementation must deny the operation without revealing whether the target object exists, unless existence is intentionally public.

## Required evidence

Maintain negative integration tests for every sensitive resource and role. Test query filters, path IDs, body IDs and nested IDs because authorization failures can occur in any one of those locations.

# **Tour-management workflow audit — Customer booking from search to confirmation**

## **Control objective**

The workflow must preserve confidentiality, integrity, availability and accountability across every transition.

## **Failure cases**

Test duplicate submissions, retries, stale screens, concurrent operators, provider timeouts, partial payments, changed tour capacity, canceled tours, reassignment and unauthorized users. Every failure must leave the database in a valid state.

## **Acceptance criteria**

Define a state diagram and invariants for the workflow. Implement transactions or atomic operations where shared resources are involved. Log security-sensitive changes and expose clear operational statuses.

# **Tour-management workflow audit — Agent booking and commission**

## **Control objective**

The workflow must preserve confidentiality, integrity, availability and accountability across every transition.

## **Failure cases**

Test duplicate submissions, retries, stale screens, concurrent operators, provider timeouts, partial payments, changed tour capacity, canceled tours, reassignment and unauthorized users. Every failure must leave the database in a valid state.

## **Acceptance criteria**

Define a state diagram and invariants for the workflow. Implement transactions or atomic operations where shared resources are involved. Log security-sensitive changes and expose clear operational statuses.

# **Tour-management workflow audit — Tour manager assignment**

## **Control objective**

The workflow must preserve confidentiality, integrity, availability and accountability across every transition.

## **Failure cases**

Test duplicate submissions, retries, stale screens, concurrent operators, provider timeouts, partial payments, changed tour capacity, canceled tours, reassignment and unauthorized users. Every failure must leave the database in a valid state.

## **Acceptance criteria**

Define a state diagram and invariants for the workflow. Implement transactions or atomic operations where shared resources are involved. Log security-sensitive changes and expose clear operational statuses.

# Tour-management workflow audit — Guide/driver operations

## Control objective

The workflow must preserve confidentiality, integrity, availability and accountability across every transition.

## Failure cases

Test duplicate submissions, retries, stale screens, concurrent operators, provider timeouts, partial payments, changed tour capacity, canceled tours, reassignment and unauthorized users. Every failure must leave the database in a valid state.

## Acceptance criteria

Define a state diagram and invariants for the workflow. Implement transactions or atomic operations where shared resources are involved. Log security-sensitive changes and expose clear operational statuses.

# Tour-management workflow audit — Vehicle capacity and scheduling

## Control objective

The workflow must preserve confidentiality, integrity, availability and accountability across every transition.

## Failure cases

Test duplicate submissions, retries, stale screens, concurrent operators, provider timeouts, partial payments, changed tour capacity, canceled tours, reassignment and unauthorized users. Every failure must leave the database in a valid state.

## Acceptance criteria

Define a state diagram and invariants for the workflow. Implement transactions or atomic operations where shared resources are involved. Log security-sensitive changes and expose clear operational statuses.

# Tour-management workflow audit — Payment and reconciliation

## Control objective

The workflow must preserve confidentiality, integrity, availability and accountability across every transition.

## Failure cases

Test duplicate submissions, retries, stale screens, concurrent operators, provider timeouts, partial payments, changed tour capacity, canceled tours, reassignment and unauthorized users. Every failure must leave the database in a valid state.

## Acceptance criteria

Define a state diagram and invariants for the workflow. Implement transactions or atomic operations where shared resources are involved. Log security-sensitive changes and expose clear operational statuses.



# Tour-management workflow audit — Cancellation and refund

## Control objective

The workflow must preserve confidentiality, integrity, availability and accountability across every transition.

## Failure cases

Test duplicate submissions, retries, stale screens, concurrent operators, provider timeouts, partial payments, changed tour capacity, canceled tours, reassignment and unauthorized users. Every failure must leave the database in a valid state.

## Acceptance criteria

Define a state diagram and invariants for the workflow. Implement transactions or atomic operations where shared resources are involved. Log security-sensitive changes and expose clear operational statuses.

# Tour-management workflow audit — Traveler document handling

## Control objective

The workflow must preserve confidentiality, integrity, availability and accountability across every transition.

## Failure cases

Test duplicate submissions, retries, stale screens, concurrent operators, provider timeouts, partial payments, changed tour capacity, canceled tours, reassignment and unauthorized users. Every failure must leave the database in a valid state.

## Acceptance criteria

Define a state diagram and invariants for the workflow. Implement transactions or atomic operations where shared resources are involved. Log security-sensitive changes and expose clear operational statuses.

# Tour-management workflow audit — Emergency and incident management

## Control objective

The workflow must preserve confidentiality, integrity, availability and accountability across every transition.

## Failure cases

Test duplicate submissions, retries, stale screens, concurrent operators, provider timeouts, partial payments, changed tour capacity, canceled tours, reassignment and unauthorized users. Every failure must leave the database in a valid state.

## Acceptance criteria

Define a state diagram and invariants for the workflow. Implement transactions or atomic operations where shared resources are involved. Log security-sensitive changes and expose clear operational statuses.

# **Tour-management workflow audit — Post-trip completion and reviews**

## **Control objective**

The workflow must preserve confidentiality, integrity, availability and accountability across every transition.

## **Failure cases**

Test duplicate submissions, retries, stale screens, concurrent operators, provider timeouts, partial payments, changed tour capacity, canceled tours, reassignment and unauthorized users. Every failure must leave the database in a valid state.

## **Acceptance criteria**

Define a state diagram and invariants for the workflow. Implement transactions or atomic operations where shared resources are involved. Log security-sensitive changes and expose clear operational statuses.

# Remediation roadmap — 0–24 hours: contain critical exposure

## Priority action

Use the finding register to assign owners, deadlines and evidence. Critical and High issues should be tracked as release blockers unless a documented compensating control is approved.

## Engineering practice

Each remediation should include code, automated tests, operational configuration and documentation. Avoid one-off patches that create a second authorization path.

## Exit criteria

A finding is closed only when the code is merged, the test passes, the deployment configuration is verified and the behavior is observable in staging.

## **Remediation roadmap — Days 2–7: close High authorization/data risks**

### **Priority action**

Use the finding register to assign owners, deadlines and evidence. Critical and High issues should be tracked as release blockers unless a documented compensating control is approved.

### **Engineering practice**

Each remediation should include code, automated tests, operational configuration and documentation. Avoid one-off patches that create a second authorization path.

### **Exit criteria**

A finding is closed only when the code is merged, the test passes, the deployment configuration is verified and the behavior is observable in staging.

# Remediation roadmap — Week 2: payment and booking integrity

## Priority action

Use the finding register to assign owners, deadlines and evidence. Critical and High issues should be tracked as release blockers unless a documented compensating control is approved.

## Engineering practice

Each remediation should include code, automated tests, operational configuration and documentation. Avoid one-off patches that create a second authorization path.

## Exit criteria

A finding is closed only when the code is merged, the test passes, the deployment configuration is verified and the behavior is observable in staging.

# Remediation roadmap — Weeks 3–4: privacy and session architecture

## Priority action

Use the finding register to assign owners, deadlines and evidence. Critical and High issues should be tracked as release blockers unless a documented compensating control is approved.

## Engineering practice

Each remediation should include code, automated tests, operational configuration and documentation. Avoid one-off patches that create a second authorization path.

## Exit criteria

A finding is closed only when the code is merged, the test passes, the deployment configuration is verified and the behavior is observable in staging.



# Remediation roadmap — Month 2: testing and DevSecOps

## Priority action

Use the finding register to assign owners, deadlines and evidence. Critical and High issues should be tracked as release blockers unless a documented compensating control is approved.

## Engineering practice

Each remediation should include code, automated tests, operational configuration and documentation. Avoid one-off patches that create a second authorization path.

## Exit criteria

A finding is closed only when the code is merged, the test passes, the deployment configuration is verified and the behavior is observable in staging.

# Remediation roadmap — Month 3: observability and resilience

## Priority action

Use the finding register to assign owners, deadlines and evidence. Critical and High issues should be tracked as release blockers unless a documented compensating control is approved.

## Engineering practice

Each remediation should include code, automated tests, operational configuration and documentation. Avoid one-off patches that create a second authorization path.

## Exit criteria

A finding is closed only when the code is merged, the test passes, the deployment configuration is verified and the behavior is observable in staging.

# Remediation roadmap — Quarterly: penetration testing

## Priority action

Use the finding register to assign owners, deadlines and evidence. Critical and High issues should be tracked as release blockers unless a documented compensating control is approved.

## Engineering practice

Each remediation should include code, automated tests, operational configuration and documentation. Avoid one-off patches that create a second authorization path.

## Exit criteria

A finding is closed only when the code is merged, the test passes, the deployment configuration is verified and the behavior is observable in staging.

# Remediation roadmap — Quarterly: backup restore drills

## Priority action

Use the finding register to assign owners, deadlines and evidence. Critical and High issues should be tracked as release blockers unless a documented compensating control is approved.

## Engineering practice

Each remediation should include code, automated tests, operational configuration and documentation. Avoid one-off patches that create a second authorization path.

## Exit criteria

A finding is closed only when the code is merged, the test passes, the deployment configuration is verified and the behavior is observable in staging.

# Remediation roadmap — Ongoing: dependency and secret management

## Priority action

Use the finding register to assign owners, deadlines and evidence. Critical and High issues should be tracked as release blockers unless a documented compensating control is approved.

## Engineering practice

Each remediation should include code, automated tests, operational configuration and documentation. Avoid one-off patches that create a second authorization path.

## Exit criteria

A finding is closed only when the code is merged, the test passes, the deployment configuration is verified and the behavior is observable in staging.

# Remediation roadmap — Final production readiness gate

## Priority action

Use the finding register to assign owners, deadlines and evidence. Critical and High issues should be tracked as release blockers unless a documented compensating control is approved.

## Engineering practice

Each remediation should include code, automated tests, operational configuration and documentation. Avoid one-off patches that create a second authorization path.

## Exit criteria

A finding is closed only when the code is merged, the test passes, the deployment configuration is verified and the behavior is observable in staging.

# **Appendix — Route inventory and authorization review**

## **Purpose**

This appendix is intended to become a living engineering artifact rather than a static document.

## **Recommended content**

Record endpoint/module, owner, authentication requirement, permission, object-level rule, sensitive data handled, rate limit, validation schema, audit event, tests and monitoring.

## **Audit evidence**

The source review identified 59 route files, 86 controllers and 46 model files in the supplied project inventory. The breadth of the system makes automated inventory and regression testing essential.

# **Appendix — Sensitive-data inventory**

## **Purpose**

This appendix is intended to become a living engineering artifact rather than a static document.

## **Recommended content**

Record endpoint/module, owner, authentication requirement, permission, object-level rule, sensitive data handled, rate limit, validation schema, audit event, tests and monitoring.

## **Audit evidence**

The source review identified 59 route files, 86 controllers and 46 model files in the supplied project inventory. The breadth of the system makes automated inventory and regression testing essential.



# **Appendix — Dependency review**

## **Purpose**

This appendix is intended to become a living engineering artifact rather than a static document.

## **Recommended content**

Record endpoint/module, owner, authentication requirement, permission, object-level rule, sensitive data handled, rate limit, validation schema, audit event, tests and monitoring.

## **Audit evidence**

The source review identified 59 route files, 86 controllers and 46 model files in the supplied project inventory. The breadth of the system makes automated inventory and regression testing essential.

# **Appendix — Security test catalogue**

## **Purpose**

This appendix is intended to become a living engineering artifact rather than a static document.

## **Recommended content**

Record endpoint/module, owner, authentication requirement, permission, object-level rule, sensitive data handled, rate limit, validation schema, audit event, tests and monitoring.

## **Audit evidence**

The source review identified 59 route files, 86 controllers and 46 model files in the supplied project inventory. The breadth of the system makes automated inventory and regression testing essential.

# Appendix — Sample RBAC matrix

## Purpose

This appendix is intended to become a living engineering artifact rather than a static document.

## Recommended content

Record endpoint/module, owner, authentication requirement, permission, object-level rule, sensitive data handled, rate limit, validation schema, audit event, tests and monitoring.

## Audit evidence

The source review identified 59 route files, 86 controllers and 46 model files in the supplied project inventory. The breadth of the system makes automated inventory and regression testing essential.

# **Appendix — Booking state-machine checklist**

## **Purpose**

This appendix is intended to become a living engineering artifact rather than a static document.

## **Recommended content**

Record endpoint/module, owner, authentication requirement, permission, object-level rule, sensitive data handled, rate limit, validation schema, audit event, tests and monitoring.

## **Audit evidence**

The source review identified 59 route files, 86 controllers and 46 model files in the supplied project inventory. The breadth of the system makes automated inventory and regression testing essential.

# **Appendix — Payment invariant checklist**

## **Purpose**

This appendix is intended to become a living engineering artifact rather than a static document.

## **Recommended content**

Record endpoint/module, owner, authentication requirement, permission, object-level rule, sensitive data handled, rate limit, validation schema, audit event, tests and monitoring.

## **Audit evidence**

The source review identified 59 route files, 86 controllers and 46 model files in the supplied project inventory. The breadth of the system makes automated inventory and regression testing essential.

# **Appendix — Production launch checklist**

## **Purpose**

This appendix is intended to become a living engineering artifact rather than a static document.

## **Recommended content**

Record endpoint/module, owner, authentication requirement, permission, object-level rule, sensitive data handled, rate limit, validation schema, audit event, tests and monitoring.

## **Audit evidence**

The source review identified 59 route files, 86 controllers and 46 model files in the supplied project inventory. The breadth of the system makes automated inventory and regression testing essential.

# Final audit opinion and release decision

## Overall opinion

The application has a strong feature foundation but requires a security hardening and reliability phase before it should be treated as a production-grade tour management platform. The highest-value improvement is to make authorization and business invariants uniform and testable.

## Remediated in supplied audit copy

The supplied copy hardens public operational endpoints, protects system settings/database routes, reduces global request body limits, adds dedicated authentication/reset rate limits, adds multipart parser limits, enforces JWT issuer/audience validation, and authenticates Socket.IO identity server-side.

## Outstanding release blockers

The primary remaining blockers are localStorage bearer-token exposure, mass assignment in privileged controllers, sensitive traveler-data protection, production fail-closed configuration, unified authorization, atomic booking capacity, payment/webhook verification and comprehensive security regression testing.

## Decision

RELEASE WITH SECURITY REMEDIATION REQUIRED. Do not represent this report as a penetration-test certificate. Close Critical/High findings and verify production infrastructure before public launch.